

project2_py\project2.py

```
1 #
2 # File: project2.py
3 #
4
5 ## top-level submission file
6
7 '''
8 Note: Do not import any other modules here.
9     To import from another file xyz.py here, type
10     import project2_py.xyz
11     However, do not import any modules except numpy in those files.
12     It's ok to import modules only in files that are
13     not imported here (e.g. for your plotting code).
14 '''
15 import numpy as np
16
17 try:
18     from .helpers import Simple1, Simple2, Simple3
19     from .local_minimizers import adam_optimizer
20 except ImportError:
21     from helpers import Simple1, Simple2, Simple3
22     from local_minimizers import adam_optimizer
23
24
25 def augmented_lagrange_ADAM(f, g, c, x0, n, count, path, params = None):
26     #Hyper terms
27     x_length = len(x0)
28
29     # ALM Terms
30     if count() < n:
31         c0 = np.array(c(x0)).flatten() # Costs 1
32         mu = np.zeros(len(c0))
33     else:
34         mu = np.array([])
35     rho = params.get('rho', 1.0)
36     gamma_rho = params.get('gamma_rho', 1.01)
37     num_inner_loops = params.get('num_inner_loops', 12)
38
39     #ADAM Terms
40     alpha = params.get('alpha', 0.32)
41     gamma = params.get('gamma', 0.95)
42     gamma_v = params.get('gamma_v', 0.9)
43     gamma_s = params.get('gamma_s', 0.999)
44     epsilon = params.get('epsilon', 1e-5)
45
46     def penalty(x, mu, rho, c): # Each penalty call costs 1
47         if len(mu) == 0:
48             return 0.0
49         constraints = np.array(c(x)).flatten()
50         return float(np.sum((np.maximum(mu + rho*constraints, 0)**2 - mu**2) / (2*rho)))
51
52     def penalty_g(x, mu, rho, c, h = 1e-5): # Each penalty_g call costs 1 + x_length
```

```

53     x_length = len(x)
54     penalty_gh = np.zeros(x_length)
55     if len(mu) == 0:
56         return penalty_gh
57     p0 = penalty(x, mu, rho, c)
58     for i in range(x_length):
59         x_h = np.copy(x)
60         x_h[i] += h
61         penalty_gh[i] = penalty(x_h, mu, rho, c) - p0 # Forward Diff Gradient Approx.
62     return penalty_gh / h
63
64     vel = np.zeros(x_length)
65     sqr = np.zeros(x_length)
66     k = 0
67     while count() < n:
68
69         x_old = path[-1]
70
71         augmented_f = lambda x: f(x) + penalty(x, mu, rho, c) # Each augmented_f call costs 2
72         augmented_g = lambda x: g(x) + penalty_g(x, mu, rho, c) # Each augmented_g call costs 3 +
x_length
73
74         cost_g = 3 + x_length if len(mu) > 0 else 2
75         evals_for_mu = 1 if len(mu) > 0 else 0
76         augmented_n = np.minimum(count() + cost_g * num_inner_loops, n - evals_for_mu)
77
78         adam_path, alpha, k, vel, sqr = adam_optimizer(
79             augmented_f, augmented_g, x_old, augmented_n, count,
80             alpha, gamma, gamma_v, gamma_s, epsilon,
81             k, vel, sqr, cost_g
82         )
83
84         x_new = adam_path[-1]
85         if len(mu) > 0 and count() < n:
86             mu = np.maximum(mu + rho * np.array(c(x_new)).flatten(), 0) # Update the Lagrange
multiplier (This costs 1)
87             path.append(x_new)
88             rho = rho * gamma_rho
89
90             if np.linalg.norm(x_new - x_old) < epsilon:
91                 break
92     return path
93
94
95
96 def quadratic_penalty_ADAM(f, g, c, x0, n, count, path, params = None):
97     if params is None:
98         params = {}
99     x_length = len(x0)
100
101     rho = params.get('rho', 1.0)
102     gamma_rho = params.get('gamma_rho', 1.01)
103     num_inner_loops = params.get('num_inner_loops', 12)
104

```

```

105 #ADAM Terms
106 alpha = params.get('alpha', 0.30)
107 gamma = params.get('gamma', 0.999)
108 gamma_v = params.get('gamma_v', 0.9)
109 gamma_s = params.get('gamma_s', 0.999)
110 epsilon = params.get('epsilon', 1e-5)
111
112 vel = np.zeros(x_length)
113 sqr = np.zeros(x_length)
114 k = 0
115
116 def penalty(x, rho, c): # Each penalty call costs 1
117     constraints = np.array(c(x)).flatten()
118     return (rho / 2.0) * np.sum(np.maximum(constraints, 0)**2)
119
120 def penalty_g(x, rho, c, h = 1e-5): # Each penalty_g call costs 1 + x_length
121     x_length = len(x)
122     penalty_gh = np.zeros(x_length)
123     p0 = penalty(x, rho, c)
124     for i in range(x_length):
125         x_h = np.copy(x)
126         x_h[i] += h
127         penalty_gh[i] = penalty(x_h, rho, c) - p0 # Forward Diff Gradient Approx.
128     return penalty_gh / h
129
130 augmented_f = lambda x: f(x) + penalty(x, rho, c) # Each augmented_f call costs 2
131 augmented_g = lambda x: g(x) + penalty_g(x, rho, c) # Each augmented_g call costs 3 + x_length
132
133 while count() < n:
134     x_old = path[-1]
135     cost_g = 3 + x_length
136     augmented_n = np.minimum(count() + cost_g * num_inner_loops, n)
137
138     adam_path, alpha, k, vel, sqr = adam_optimizer(
139         augmented_f, augmented_g, x_old, augmented_n, count,
140         alpha, gamma, gamma_v, gamma_s, epsilon,
141         k, vel, sqr, cost_g
142     )
143     x_new = adam_path[-1]
144     path.append(x_new)
145     rho = rho * gamma_rho
146     if np.linalg.norm(x_new - x_old) < epsilon:
147         break
148 return path
149
150
151
152 def optimize_with_history(f, g, c, x0, n, count, prob):
153     """
154     Args:
155         f (function): Function to be optimized
156         g (function): Gradient function for `f`
157         c (function): Function evaluating constraints
158         x0 (np.array): Initial position to start from

```

```

159     n (int): Number of evaluations allowed. Remember `f` and `c` cost 1 and `g` costs 2
160     count (function): takes no arguments are reutrns current count
161     prob (str): Name of the problem. So you can use a different strategy
162                 for each problem. `prob` can be `simple1`, `simple2`, `simple3`,
163                 `secret1` or `secret2`
164 Returns:
165     x_best (np.array): best selection of variables found
166 """
167
168 path = [x0]
169 if prob == 'simple1': ### Augmented Lagrange Method (ALM) ###
170     #return []
171     simple1_params = {
172         'rho': 1.0,
173         'gamma_rho': 1.50,
174         'num_inner_loops': 12,
175         'alpha': 0.50,
176         'gamma': 0.98,
177         'gamma_v': 0.9,
178         'gamma_s': 0.999,
179         'epsilon': 1e-8
180     }
181     #path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, simple1_params)
182     path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, simple1_params)
183
184 elif prob == 'simple2':
185     #return []
186     simple2_params = {
187         'rho': 1.0,
188         'gamma_rho': 1.5,
189         'num_inner_loops': 12,
190         'alpha': 0.10,
191         'gamma': 0.99999,
192         'gamma_v': 0.9,
193         'gamma_s': 0.999,
194         'epsilon': 1e-8
195     }
196     #path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, simple2_params)
197     path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, simple2_params)
198
199 elif prob == 'simple3':
200     #return []
201     simple3_params = {
202         'rho': 1.0,
203         'gamma_rho': 1.50,
204         'num_inner_loops': 12,
205         'alpha': 0.50,
206         'gamma': 0.99,
207         'gamma_v': 0.9,
208         'gamma_s': 0.999,
209         'epsilon': 1e-8
210     }
211     path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, simple3_params)
212     #path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, simple3_params)

```

```

213 elif prob == 'secret1':
214     #return []
215     secret1_params = {
216         'rho': 1.0,
217         'gamma_rho': 1.50,
218         'num_inner_loops': 3,
219         'alpha': 0.50,
220         'gamma': 0.99,
221         'gamma_v': 0.9,
222         'gamma_s': 0.999,
223         'epsilon': 1e-8
224     }
225     path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, secret1_params)
226     #path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, secret1_params)
227
228 elif prob == 'secret2':
229     #return []
230     secret2_params = {
231         'rho': 1.0,
232         'gamma_rho': 1.50,
233         'num_inner_loops': 2,
234         'alpha': 0.50,
235         'gamma': 0.95,
236         'gamma_v': 0.9,
237         'gamma_s': 0.999,
238         'epsilon': 1e-8
239     }
240     #path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, secret2_params)
241     path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, secret2_params)
242
243 return path
244
245
246
247
248 def optimize(f, g, c, x0, n, count, prob):
249     """
250     Wrapper for autograder that only returns the final point.
251     """
252     path = optimize_with_history(f, g, c, x0, n, count, prob)
253     return path[-1]
254
255 if __name__ == '__main__':
256     try:
257         from .plotters import plot_problem, plot_convergence, plot_constraint
258     except ImportError:
259         from plotters import plot_problem, plot_convergence, plot_constraint
260     current_problem = Simple2()
261     if current_problem.xdim == 2:
262         print(f"Rendering 2D optimization paths for {current_problem.prob}...")
263         plot_problem(current_problem, plot_size=3)
264         plot_convergence(current_problem)
265         plot_constraint(current_problem)
266

```